Project 1

EMCH 368-003

Christopher Flohr

Joud Satme

2/28/26

Introduction

The goal of this project is to design an alarm clock utilizing the tools and equipment that we have learned about up to this point in the semester. Technical requirements for the design of the alarm were to include a potentiometer to adjust time, incorporate at least one switch to control the countdown, use a seven-segment display to visibly display the countdown, and to sound an audible alarm with a piezo buzzer. This set up is a very simple, but applicable design to show how a countdown timer might perform on a small scale.

To accomplish this goal, I knew I wanted to add additional switches to better control the functions of starting, stopping, and adjusting the alarm clock countdown. The potentiometer, piezo buzzer, and seven segment display were easy to visualize and incorporate due to recent experience in other lab assignments. In terms of design layout, I wanted to keep it simple and easy for me to keep track of. I first built a circuit that included all required equipment and assigned pins to necessary inputs and outputs. Reusing some coding and circuit layout from previous exercises, including Lab 4, Lab 3, and Lab 2 material streamlined this process.

Materials

- UNO R3 Controller Board- This microcontroller provides digital and analog input/output pins, power regulation, communication. It acts as a programmable environment to control the behavior of the alarm clock.
- 830 Tie-Points Breadboard-The breadboard acts as a medium to connect electrical components for the circuit.
- Button (x3)- A switch interrupts the flow of electricity. In this setting, the switches momentarily complete a circuit that calls for a change of state in the circuit.
- Breadboard Jumper Wires-Wires electrically connect different components.
- Resistors (x7)- resistors are components that resist flow of electrical energy. In this setting the resistors are added to protect electrical components in the circuit.
- Piezo Buzzer-A small element that vibrates when it receives electricity. In this case, we use the piezo to receive a signal when the alarm hits zero, to give an audible element of the alarm clock.
- 4 Digit 7-Segment Display- A device used to show numbers. In this circuit, the 7-segment display will visibly display the countdown.
- Potentiometer- A variable resistor with three pins. The middle pin moves across the resistor to provide variable resistance. In this setting, the potentiometer's variable resistance is equated to a varying amount of time on the clock.
- USB Cable- Universal Serial Bus cable is a communication and power cable to connect the circuit to a computer. In this project, the USB cable uploads the program code to the Arduino board and provides power to the circuit.

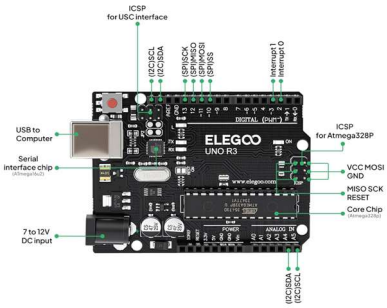


Figure 1- UNO R3 Controller Board [1]

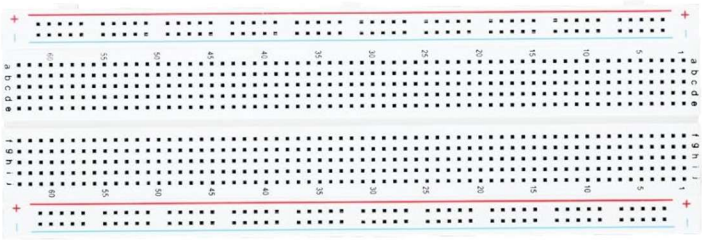


Figure 2- 830 Tie-Points Breadboard [1]



Figure 3- Button (x3) [1]



Figure 4- Breadboard Jumper Wires [1]

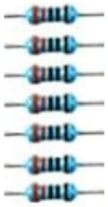


Figure 5- Resistors (x7) [1]



Figure 6- Piezo Buzzer [1]



Figure 7-4 Digit 7-Segment Display [1]



Figure 8- Potentiometer [1]



Figure 9- USB Cable [1]

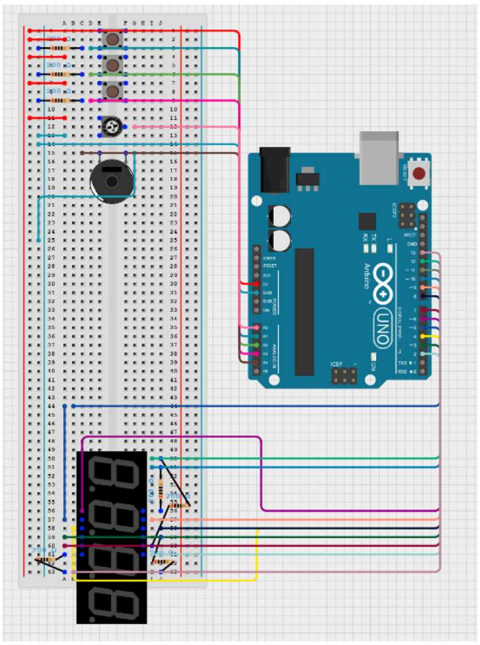


Figure 10-Schematic Representation of Electrical Circuit [2]

Hardware Design Procedure

Building the circuit for the alarm clock was streamlined by reusing concepts from previous lab assignments. Setting up the 4 digit 7-segment display followed the procedure listed in lab 4. Then I connected the three switches to analog pins, with each switch grounded with resistors. Next, I placed the potentiometer connected to power and ground with the middle pin connected to A0 pin. Finally, I added the piezo to connect to pin A4 and ground.

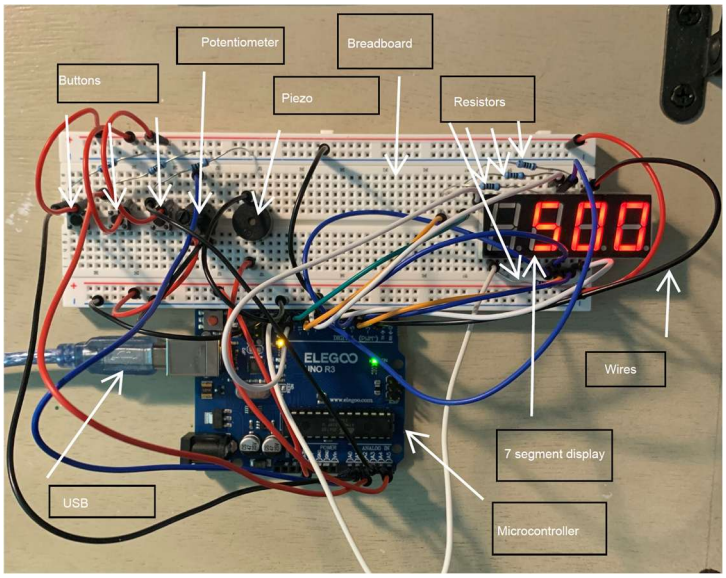


Figure 11-Physical Circuit

Building the circuit:

1. Connect a red wire to the 5V pin and put the other end in one of the long (+) bus lines in the breadboard. If needed, connect opposite (+) bus line to power.
2. Connect a black wire to the ground pin to the adjacent long (-) bus line on the breadboard. If needed, connect opposite (-) bus line to ground.
3. Place the 4-digit 7-segment display on the breadboard.
4. Connect pins E, D, DP, C, G, A, F, B to digital pins 6,5,4,3,7,9,8,2, respectively.
5. For pins, D1, D2, D3, and D4, first wire them to 1k resistors then from the resistors, wire to digital pins 10, 11, 12, 13, respectively. [4]
6. Place 3 switches on the breadboard. Connect power to each switch and ground the other end with 1K resistors.
7. Connect the buttons to A1, A2, A3 pins to set up button functions.

8. Place a potentiometer on the board. Wire power to one end, ground the other end. Wire the middle pin to A0.
9. Place piezo on the board, connect one end to ground. Connect the other end to A4.

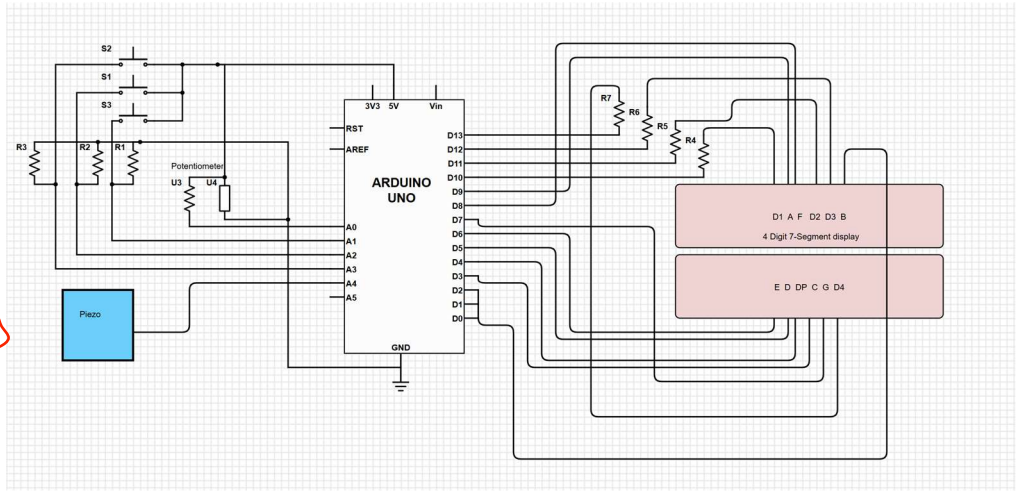


Figure 12-Electrical Schematic [2]

Algorithm and Code

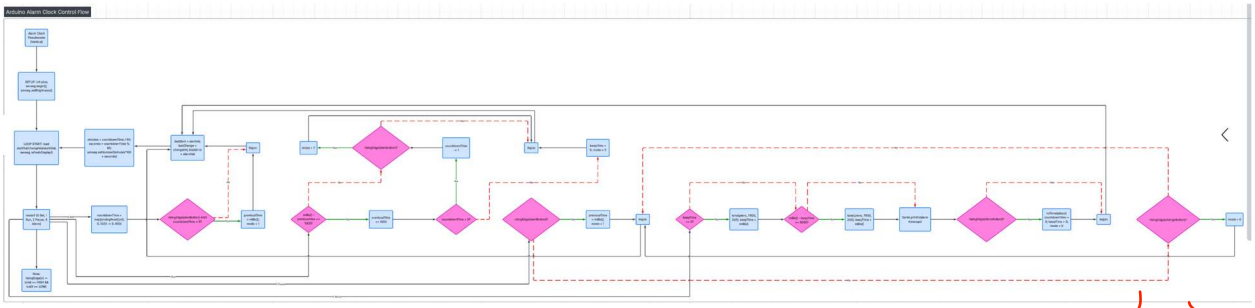


Figure 13-Coding Flowchart [3]

great Job!

Setting up the code to control the alarm clock was the most challenging part of the project. Initially, the code was set up with “while” loops and “if-else” statements. The coding never worked as intended and I needed assistance from peers. The final coding worked when

the structure was switched to be based on the different “modes” of the alarm clock using the “switch” and “case” functions. The last adjustment that was made was the inclusion of the “rising edge” which checks if the state of the button has changed, rather than just detecting when the button is being held down. This allowed for adjusting the time to happen without simultaneously holding the Change Button down. The explanation of the coding is as follows:

- Add in the 7-segment library for Arduino. Set up constant integers for the potentiometer, start button, change button, piezo, and alarm cutout. Set up initial readings for the state of the circuit. (Figure 14) [4]
- Void setup begins serial communication at 9600 baud. Sets buttons to be inputs and the piezo to be an output. Code reused from the 7-segment display lesson sets up the 7-segment. (Figure 15) [4]
- Void loop begins. Refresh the 7-segment display and begin reading inputs. Case 0 is established as the time mode. Read the potentiometer and map the value to a time between 0 and 5 minutes. Sets the countdown time. Rising edge on the start button changes the mode to running mode. (Figure 15)
- Case 1 is running mode. If loop counts down from set time, if countdown time = 0, switches to mode 3, alarming. If rising edge on the start stop button pauses the timer, switch to case 2, paused mode. While paused, give the option to resume countdown, or change time based on the change button. (Figure 16)
- Case 3 is the alarming state. Set the piezo to output with the “tone” function, based on the Sound lab. Using an if loop to set the buzzer timing sequence to have a gap of 3 seconds between beeps. While alarming, print to the Serial Monitor “The alarm is

on. Please press the alarm button to turn off." Once Rising edge on the alarm cutoff button, the "noTone" function is used to stop the piezo and mode is returned to 0. (Figure 17)

- Outside the mode functions, keep track of state of buttons, format time left on countdown, and display the number on the 7-segment display before repeating the lines of code. (Figure 17)

Conclusion

The result of the project did result in a functional alarm clock that meets the requirements laid out in the instructions. There are aspects of the circuit that were more challenging than expected. The physical layout of components proved to be hard to keep track of and took up a lot of space. In future projects, I hope to be more efficient in spacing and utilizing available digital/analog pins.

For me, the most difficult part of the project was getting functional code that worked with my circuit. My initial idea of using loops instead of switch state coding did not work because the loops keep reading the same lines of code. Additionally, when trying to add a three second delay to the piezo beeps, I tried adding the delay (3000) function directly at the end of the code. This caused my circuit to run far behind because it added the delay after reading through the code each time.

My knowledge and familiarity with the Arduino and components are improved and I am getting more comfortable with the concepts and equipment. This project gave me new ideas

that I look to implement going forward. Many of these lessons will be applied to the final Car Project.

Appendix

Alarm_clock_code.ino

```
#include "SevSeg.h"
SevSeg sevseg;
//set constants
const int pot = A0; //potentiometer
const int startButton = A3; //stop/start button
const int changeButton = A2; // when stopped, adjust countdown time
const int piezo = A4; //buzzer
const int alarmButton = A1; //alarm cutout

int potValue = 0;

int changeVal = 0; //High/Low from alarm change button when timing = false
int startVal = 0; //High/Low reading from stop start button
int lastChange = 0; // extra variable to define state of countdown
int lastStart = 0;  //0 when low, and 1 when high for both change and start
int lastAlarm = 0;

long beepTime = 0;

long potTime = 0;
long countdownTime = 0;

int minutes = 0;
int seconds = 0;
long previousTime = 0; //track millis

int mode = 0; //mode 0 set time, mode 1 run, mode 2 pause,  mode 3 alarm

void setup() {
  Serial.begin(9600);
  pinMode(startButton, INPUT);
  pinMode(changeButton, INPUT);
  pinMode(alarmButton, INPUT);
```

Figure 14- Code (pg. 1)

Alarm_clock_code.ino

```
  pinMode(alarmButton, INPUT);
  pinMode(piezo, OUTPUT);

  byte numDigits = 4;
  byte digitPins[] = {10, 11, 12, 13};
  byte segmentPins[] = {9, 2, 3, 5, 6, 8, 7, 4};
  bool resistorsonSegments = true;
  byte hardwareConfig = COMMON_CATHODE;
  sevseg.begin(hardwareConfig, numDigits, digitPins, segmentPins, resistorsonSegments, false);
  sevseg.setBrightness(100);

}

void loop() {
  startVal = digitalRead(startButton);
  changeVal = digitalRead(changeButton);
  int alarmVal = digitalRead(alarmButton);
  sevseg.refreshDisplay();

  switch(mode){
    case 0: //time mode

      //if(changeVal == HIGH && lastChange == LOW){ //change button activated
        potTime = map(analogRead(pot), 0, 1023, 0, 300); // 0-5 minutes
        countdownTime = potTime;
      //}

      if(startVal == HIGH && lastStart == LOW && countdownTime > 0){
        previousTime = millis();
        mode = 1;
      }
    break;
```

Figure 15- Code (pg 2)

Alarm_clock_code.ino

```
67    case 1:  // running mode
68      if(millis() - previousTime >= 1000){ //checks to see if 1 second has passed or not
69        previousTime = previousTime + 1000;
70        if(countdownTime > 0){
71          countdownTime = (countdownTime - 1);
72        }
73
74        else{
75          beepTime = 0;
76          mode = 3;
77        }
78      }
79
80      if(startVal == HIGH && lastStart == LOW){ // this pauses the clock
81        mode = 2;
82      }
83    break;
84
85    case 2: //when paused
86      if(startVal == HIGH && lastStart == LOW){ // resume
87        previousTime = millis();
88        mode = 1;
89      }
90
91      if(changeVal == HIGH && lastChange == LOW){ //while paused, mode 0 to change time
92        mode = 0;
93      }
94    break;
95
96    case 3: //alarming
97
98      if(beepTime == 0){
```

Figure 16- Code (pg 3)

Alarm_clock_code.ino

```
97
98      if(beepTime == 0){
99        tone(piezo, 2500, 250);
100       beepTime = millis();
101     }
102
103     if (millis() - beepTime >= 3000){
104       tone(piezo, 2500, 250);
105       beepTime = millis(); //resets the count
106     }
107
108     Serial.println("The alarm is on. Please press the alarm button to turn off.");
109
110
111     if(alarmVal == HIGH && lastAlarm == LOW){
112       noTone(piezo);
113       countdownTime = 0;
114       beepTime = 0;
115       mode = 0;
116     }
117
118   break;
119   }
120   lastStart = startVal;
121   lastChange = changeVal;
122   lastAlarm = alarmVal;
123   minutes = countdownTime / 60;
124   seconds = countdownTime % 60;
125   int number = (minutes * 100) + seconds;
126   sevseg.setNumber(number);
127
128
129 }
```

Figure 17- Code (pg 4)

References

1. ELEGOO UNO, “ELEGOO UNO Project Super Starter Kit with Tutorial and UNO R3 Board Compatible with Arduino IDE,” Amazon [Online]. Available: https://www.amazon.com/dp/B01D8KOZF4?maas=maas_adg_D460F095D342E4F0FE45D237008B6DCD_afap_abs&ref_=aa_maas&tag=maas.

2. Cirkit Designer, “Cirkit Designer – Online Circuit Design Tool.” [Online]. Available: https://app.cirkitdesigner.com/project.

3. Lucidchart, “Lucid Flowchart Design Tool.” [Online]. Available: https://lucid.app/lucidchart/0dff2584-2307-47b2-980b-e0e7e64beae2/edit?beaconFlowId=59654C19D476AA25&invitationId=inv_959e0588-034d-4085-85f1-54961ec72834&page=0_0#.

4. T. Russo, “EMCH 368 Mechatronics Lab 4 LCD”, EMCH 368, University of South Carolina, edited 6/23/24

5. T. Russo, “EMCH 368 Mechatronics Lab 3 Sound”, EMCH 368, University of South Carolina, edited 6/23/24

6. J. Satme, “EMCH 368 Mechatronics Lab 1 Introduction to Microcontrollers and Circuits”, EMCH 368, University of South Carolina, edited 1/6/25.